# Les Modèles Algorithmiques Fondamentaux sur Structures à Accès Direct

**Thème principal :** Conception d'algorithmes de parcours, mécanismes d'accès aux collections, gestion des bornes et recherche de première occurrence

**Synthèse exécutive :** Ce cours présente les schémas réutilisables fondamentaux de la sous-discipline R1.01 (Initiation à l'algorithmique) indispensable pour réussir les devoirs et les SAE du BUT Informatique. Avant d'écrire la moindre ligne de code ou de pseudo-code, l'informaticien doit analyser la structure de données sous-jacente et déterminer son mécanisme d'accès (Direct, Séquentiel ou Dynamique). Le cours détaille trois modèles canoniques : (1) Le parcours complet avec traitement systématique, (2) Le parcours complet avec traitement conditionné, et (3) La recherche de première occurrence. Une attention toute particulière est portée à la rigueur de la gestion des bornes (indices de 0 à N-1 pour éviter les erreurs de dépassement 'out of bounds'), aux doubles conditions d'arrêt obligatoires dans les recherches, et aux deux variantes de la recherche selon que l'on privilégie la simplicité de codage ou la conservation de l'indice exact.

## 1. Les Mécanismes d'Accès aux Collections de Données

Avant d'écrire un algorithme, l'étudiant doit impérativement s'interroger sur la nature des accès autorisés par la structure manipulée. Une structure à accès direct (comme un tableau à N cases ou une chaîne de caractères) associe chaque valeur à une position mémoire calculable immédiatement à partir d'un indice i (variant de 0 à N-1). À l'inverse, un flux séquentiel (comme un fichier texte brut, un CSV lu en ligne par ligne ou une liste chaînée simple) requiert trois opérations minimales : (1) se positionner au début, (2) lire/passer à l'élément suivant, et (3) tester la fin de structure. Les modèles présentés dans ce cours s'appliquent aux structures à accès direct. Appliquer une logique d'accès direct (par exemple demander l'élément à l'index i) sur un flux séquentiel est une faute de logique majeure.

- Structure de données : Collection regroupant un ensemble d'éléments de même type (ex: tableaux, chaînes de caractères).
- Accès Direct : Capacité d'atteindre instantanément n'importe quel élément via un indice ou une clé, sans visiter les éléments précédents.
- Accès Séquentiel : Contrainte d'accès imposant de démarrer au début et de progresser élément par élément jusqu'à la fin.
- Accès Dynamique : Structure combinant à la fois les facilités d'accès direct et séquentiel.

***Cas concret enseignant :*** *Le professeur souligne l'erreur classique en partiel : tenter d'accéder directement à un indice T[i] sur un fichier CSV brut ou un fichier texte. Comme un fichier est parcouru séquentiellement ligne à ligne, une telle instruction invalide immédiatement la conception de l'algorithme.*

## 2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Le modèle de parcours complet systématique est mis en œuvre lorsqu'aucune exclusion n'est tolérée entre la borne initiale et la borne finale. La structure naturelle est la boucle bornée 'Pour', car le nombre d'itérations est connu avant d'entrer dans la boucle (nombre de cases = Fin - Début + 1). Dans un tableau de N éléments en notation standard, le premier élément est à l'indice 0 et le dernier à l'indice N-1. L'algorithme standard s'écrit : Pour i allant de 0 à N - 1 Faire Traiter(Collection[i]) Fin Pour L'erreur fatale (piège de partiel) consiste à faire aller l'indice i jusqu'à N, déclenchant une erreur de débordement d'indice (out of bounds).

- Objectif : Appliquer un traitement à 100% des éléments situés entre une borne de début et une borne de fin.
- Structure itérative : Boucle bornée (Pour / For).
- Gestion stricte des bornes : Pour une collection de N éléments, les indices s'étendent de 0 à N - 1.

***Cas concret enseignant :*** *Exemple d'illustration : Afficher toutes les valeurs d'un tableau d'entiers ou passer en majuscules chaque caractère d'une chaîne. La fonction de traitement s'exécute de manière indifférenciée sur chaque élément rencontré.*

## 3. Second Modèle : Le Parcours Complet avec Traitement Conditionné

Ce modèle conserve l'exhaustivité du parcours (tous les éléments de la zone sont inspectés de la borne de début à la borne de fin), mais le traitement principal est protégé par un filtre conditionnel. L'algorithme s'exprime ainsi : Pour i allant de 0 à N - 1 Faire Si Propriété(Collection[i]) Alors Traiter(Collection[i]) Fin Si Fin Pour La condition peut être simple (ex: T[i] > 0) ou résulter de tests combinés (ex: T[i] > 0 ET T[i] mod 2 = 0). Elle peut également, dans des algorithmes plus avancés, comparer l'élément courant avec ses voisins (ex: T[i] > T[i-1]).

- Objectif : Examiner l'ensemble d'une structure mais ne déclencher le traitement que sur les éléments satisfaisant un test logique.
- Structure : Une boucle bornée 'Pour' englobant une structure alternative 'Si ... Alors'.
- Conditions complexes : Le test peut porter sur la valeur courante ou sur des combinaisons d'éléments adjacents.

***Cas concret enseignant :*** *L'analogie du contrôle qualité sur une chaîne de montage : Chaque pièce défile devant le contrôleur (parcours complet systématique d'inspection), mais le contrôleur n'appose un tampon de rebut (traitement) que si la pièce présente un défaut (traitement conditionné).*

## 4. Troisième Modèle : La Recherche de Première Occurrence

Contrairement aux parcours complets, une recherche de première occurrence s'appuie sur une itération non bornée ('Tant que') car le nombre d'itérations est imprévisible. L'algorithme adopte une posture pessimiste : on part du principe que l'élément n'existe pas (trouve <- Faux). Un algorithme de recherche valide DOIT impérativement posséder DEUX conditions d'arrêt : 1. La recherche est fructueuse (trouve = Vrai). 2. La recherche a échoué après exploration de toute la collection (fin de structure atteinte). Variante 1 (Rupture) : Utilise une sortie anticipée dès que la condition est vérifiée, ce qui permet à l'indice i de rester positionné pile sur l'élément trouvé. Variante 2 (Tant que classique) : La boucle s'écrit `Tant que (i < N et NON trouve) Faire`. Si la propriété est vérifiée, trouve devient Vrai et l'indice i est incrémenté une dernière fois, ce qui nécessite un réajustement d'indice si l'on souhaite réutiliser la position.

- Objectif : Localiser le tout premier élément vérifiant une propriété et interrompre l'exploration immédiatement.
- Pessimisme algorithmique : Initialisation d'un booléen 'trouve' à Faux avant d'engager la recherche.
- Double condition d'arrêt obligatoire : (1) A-t-on épuisé toute la structure ? (2) A-t-on trouvé l'élément recherché ?
- Variante 1 (Rupture immédiate) : S'arrête exactement sur l'indice de l'élément trouvé.
- Variante 2 (Tant que global) : Regroupe les conditions d'arrêt en tête de boucle, simplifiant la syntaxe de codage.

***Cas concret enseignant :*** *Le professeur insiste : 'En informatique, soyez pessimistes ! Partez du principe que ce que vous cherchez n'est pas là, et modifiez votre indicateur dès que vous le trouvez.' Il rappelle qu'écrire un algorithme de recherche avec une seule condition d'arrêt est une faute grave sanctionnée en évaluation.*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Est-ce qu'on peut appliquer le parcours complet avec traitement systematic sur une structure à accès dynamique ?
**Réponse du professeur :** Oui, tout à fait. Une structure à accès dynamique offre à la fois des fonctionnalités séquentielles et directes. On peut donc parfaitement exploiter ses capacités d'accès direct pour y appliquer un parcours complet systématique ou conditionné.

**Question étudiant :** Pourquoi s'embêter à présenter deux variantes différentes pour la recherche de première occurrence ?
**Réponse du professeur :** Elles répondent à deux besoins différents. La variante 1 à rupture immédiate s'arrête exactement sur l'indice de l'élément trouvé, ce qui est idéal si le problème exige de récupérer sa position exacte. La variante 2 regroupe les conditions d'arrêt dans la tête du 'Tant que', ce qui offre une structure plus propre et plus facile à traduire directement dans certains langages de programmation.

**Question étudiant :** Comment fait-on si la condition dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
**Réponse du professeur :** Le modèle reste un parcours complet conditionné. Simplement, la condition logique portée par le 'Si' adapte son expression en comparant l'élément courant T[i] à T[i-1] ou T[i+1]. Il faut juste faire attention à adapter les bornes (par exemple commencer à i = 1) pour ne pas sortir du tableau lors du test de l'élément précédent.

**Question étudiant :** Si l'on cherche TOUS les indices correspondant à une valeur (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
**Réponse du professeur :** Il faut utiliser le Modèle 2 : Parcours complet avec traitement conditionné ! La recherche de première occurrence (Modèle 3) s'arrête dès qu'elle trouve le premier '5'. Pour tous les trouver, il faut parcourir l'intégralité du tableau de la borne 0 à N-1.

## ■ Exercice & Démonstration d'Application

**Énoncé :** Écrire l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en utilisant le modèle algorithmique approprié.

**Résolution pas à pas :**
1. Analyse du problème : Il faut examiner l'ensemble des éléments d'une structure à accès direct (tableau T de N cases). L'affichage (traitement) ne doit s'exécuter que si la valeur est strictement supérieure à 0. Il s'agit donc du 'Parcours complet avec traitement conditionné'. 2. Identification des bornes : Le tableau comprend N cases indicées de 0 à N - 1. Borne inférieure = 0, borne supérieure = N - 1. 3. Choix de la boucle : Une boucle bornée 'Pour i allant de 0 à N - 1'. 4. Condition d'exécution : 'Si T[i] > 0 Alors'. 5. Action : 'Afficher(T[i])'.

**Résultat : Algorithme AffichagePositifs Données : T : Tableau d'entiers, N : Entier Variables : i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire : Cet algorithme garantit un parcours exhaustif de 0 à N-1 sans risque de débordement mémoire et applique le filtre logique strictement demandé.**

## ■ Auto-Évaluation Flash d'Examen

**Q : Quel modèle algorithmique doit-on utiliser pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?**

*R : La recherche de première occurrence (Modèle 3), car il est inutile de parcourir le reste du tableau dès que la première valeur 10 est localisée.*

**Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il égal à N - 1 et non à N ?**

*R : Parce qu'en informatique standard, la numérotation des indices commence à 0. Un tableau de N éléments possède donc ses cases numérotées de 0 à N - 1. Accéder à l'indice N provoque une erreur de dépassement de capacité (out of bounds).*

**Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence ?**

*R : Elle doit obligatoirement comporter DEUX conditions d'arrêt minimales : (1) La découverte de l'élément recherché (recherche fructueuse) et (2) L'atteinte de la fin de la structure de données (recherche infructueuse).*